CS 4782 Final Project Report

Re-Implementation of "LoRA: Low-Rank Adaptation of Large Language Models"

Ajay Tadinada (at663), Dan Wei Zuo (dz267), Mericel Tao (mst223)
Cornell University
GitHub Repository: https://github.com/Ajay-T/Re-implementation-of-LoRA

May 12, 2026

1 Introduction

Full Fine Tuning (FFT) of Large Language Models (LLMs) demands significant VRAM and extended training times because it updates every model parameter. For models like RoBERTa or GPT-2, this means training over 100 million parameters for each downstream task. To address this, our project re-implements and evaluates the methods proposed in *"LoRA: Low-Rank Adaptation of Large Language Models"* by Hu et al.

The primary contribution of the paper is a parameter efficient fine tuning method. By injecting trainable low-rank decomposition matrices ($A \in \mathbb{R}^{r\times k}$ and $B \in \mathbb{R}^{d\times r}$) into the dense layers of pre-trained transformer models while freezing the original pre-trained weights ($W_0 \in \mathbb{R}^d \times k$), the authors hypothesize that downstream task performance can match FFT when using $h = W_0x + \Delta Wx = W_0x + BAx$ for the forward pass. Furthermore, restricting the rank ($r \ll d_{model}$) dramatically reduces the number of trainable parameters, yielding lower VRAM consumption and faster training times. Our motivation is to verify these claims across diverse transformer workloads and test LoRA's generalizability beyond the original language model settings.

2 Chosen Result

We aimed to reproduce the original paper's central claim: LoRA can achieve comparable downstream task performance to FFT on GLUE benchmarks and generation tasks while reducing trainable parameters by orders of magnitude (paper reports 10,000×). We specifically targeted the replication of parameter reduction metrics, peak VRAM usage (3× lower compared to FFT), training time (original authors saw 25% reduction using LoRA with GPT-3), and validation accuracy (as shown in Figures 1 and 2).

Model & Method	# Trainable Parameters	E2E NLG Challenge				
		BLEU	NIST	MET	ROUGE-L	CIDEr
GPT-2 M (FT)*	354.92M	68.2	8.62	46.2	71.0	2.47
GPT-2 M (Adapter^{L})*	0.37M	66.3	8.41	45.0	69.8	2.40
GPT-2 M (Adapter^{L})*	11.09M	68.9	8.71	46.1	71.3	2.47
GPT-2 M (Adapter^{H})	11.09M	$67.3_{\pm.6}$	$8.50_{\pm.07}$	$46.0_{\pm.2}$	$70.7_{\pm.2}$	$2.44_{\pm.01}$
GPT-2 M (FT^{Top2})*	25.19M	68.1	8.59	46.0	70.8	2.41
GPT-2 M (PreLayer)*	0.35M	69.7	8.81	46.1	71.4	2.49
GPT-2 M (LoRA)	0.35M	**$70.4_{\pm.1}$**	**$8.85_{\pm.02}$**	**$46.8_{\pm.2}$**	**$71.8_{\pm.1}$**	**$2.53_{\pm.02}$**

Figure 1: Original paper NLG metrics for GPT-2 Medium

We chose this because it represents the core value proposition of LoRA, allowing the fine tuning of large models in compute constrained environments. Additionally, we designed an independent extension to test whether LoRA's parameter efficiency and accuracy preservation generalize to image classification using a Vision Transformer (ViT).

Model & Method	# Trainable Parameters	MNLI	SST-2	MRPC	CoLA	QNLI	QQP	RTE	STS-B	Avg.
RoB$_{base}$ (FT)*	125.0M	**87.6**	94.8	90.2	**63.6**	92.8	**91.9**	78.7	91.2	86.4
RoB$_{base}$ (BitFit)*	0.1M	84.7	93.7	**92.7**	62.0	91.8	84.0	81.5	90.8	85.2
RoB$_{base}$ (Adpt^{D})*	0.3M	$87.1_{\pm.0}$	$94.2_{\pm.1}$	$88.5_{\pm1.1}$	$60.8_{\pm.4}$	$93.1_{\pm.1}$	$90.2_{\pm.0}$	$71.5_{\pm2.7}$	$89.7_{\pm.3}$	84.4
RoB$_{base}$ (Adpt^{D})*	0.9M	$87.3_{\pm.1}$	$94.7_{\pm.3}$	$88.4_{\pm.1}$	$62.6_{\pm.9}$	$93.0_{\pm.2}$	$90.6_{\pm.0}$	$75.9_{\pm2.2}$	$90.3_{\pm.1}$	85.4
RoB$_{base}$ (LoRA)	0.3M	$87.5_{\pm.3}$	**$95.1_{\pm.2}$**	$89.7_{\pm.7}$	$63.4_{\pm1.2}$	**$93.3_{\pm.3}$**	$90.8_{\pm.1}$	**$86.6_{\pm.7}$**	**$91.5_{\pm.2}$**	**87.2**

Figure 2: Original paper NLG metrics for RoBERTa base

3 Methodology

Model Architecture and Implementation: Our re-implementation wraps existing PyTorch `nn.Linear` modules in a custom `LoRALinear` class. The original weights are frozen, and two trainable matrices are initialized: A (projects input down to rank r) and B (projects back up to the original dimension). An injection script dynamically walks the model tree and replaces matching linear layers, specifically targeting the attention query (W_q) and value (W_v) projection layers.

Datasets and Tasks: We selected models to test LoRA across diverse architectures:

- **RoBERTa-base (Encoder only):** Evaluated on SST-2 (binary sentiment classification) and MNLI (natural language inference). These are standard GLUE NLU tasks used to establish baseline parity with FFT.
- **GPT-2 Small/Medium (Decoder only):** Evaluated on the E2E NLG dataset (generating restaurant descriptions from meaning representations) to test LoRA's efficacy on natural language generation.
- **ViT (Vision Transformer):** Evaluated on CIFAR-10 (image classification). This was our novel extension to test generalizability.

Modifications and Challenges: A key obstacle arose when adapting the code for GPT-2. Hugging Face's implementation groups the calculation of Q, K, and V into a single `c_attn` layer. We had to modify our injection script to separate these components, compute the LoRA matrices for Q and V, and concatenate them back. Additionally, due to compute constraints, we scaled down the GPT-2 experiments, utilizing GPT-2 Small/Medium instead of the Medium/Large models used in the original paper. We adjusted hyperparameters accordingly, using $r = 4, \alpha = 32$ for Medium (5 epochs) and $r = 8, \alpha = 32$ for Small (10 epochs) to boost accuracy. All training was conducted on T4 and A100 GPUs.

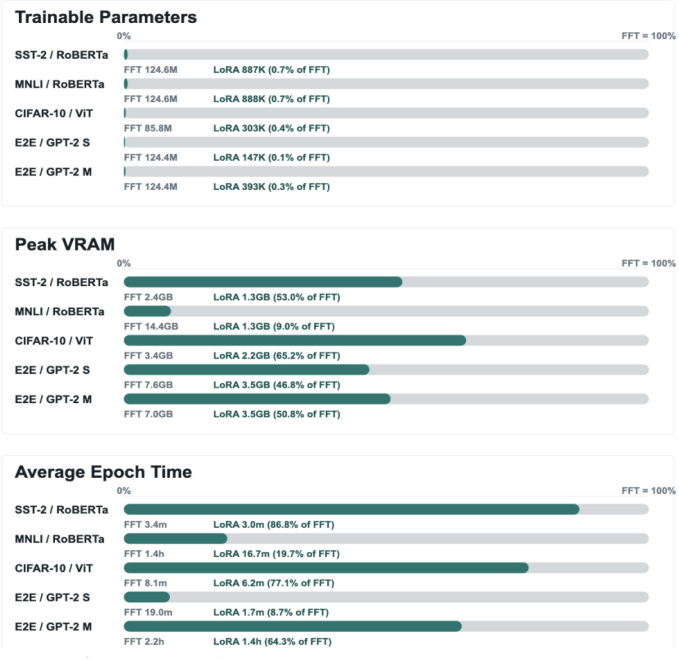

Figure 3: Efficiency comparison between our FFT and LoRA implementations

Model	Metric	Paper	Ours	Δ
Classification — FFT				
ViT (CIFAR-10)	Accuracy	–	0.9868	–
RoBERTa (MNLI)	Accuracy	87.6	87.7	**+0.1**
RoBERTa (SST-2)	Accuracy	94.8	93.9	**-0.9**
Classification — LoRA				
ViT (CIFAR-10)	Accuracy	–	0.9830	–
RoBERTa (MNLI)	Accuracy	87.5	85.9	**-1.6**
RoBERTa (SST-2)	Accuracy	95.1	94.0	**-1.1**
Generation (E2E) — FFT				
GPT-2 Small	BLEU	–	53.68	–
	NIST	–	6.02	–
	METEOR	–	69.58	–
	ROUGE-L	–	62.55	–
	CIDEr	–	1.17	–
GPT-2 Med.	BLEU	68.2	54.11	**-14.09**
	NIST	8.62	6.02	**-2.60**
	METEOR	46.2	69.10	**+22.90**
	ROUGE-L	71.0	62.03	**-8.97**
	CIDEr	2.47	1.23	**-1.24**
Generation (E2E) — LoRA				
GPT-2 Small	BLEU	–	52.98	–
	NIST	–	3.11	–
	METEOR	–	48.99	–
	ROUGE-L	–	62.84	–
	CIDEr	–	1.04	–
GPT-2 Med.	BLEU	70.4	53.68	**-16.72**
	NIST	8.85	6.05	**-2.80**
	METEOR	46.8	65.18	**+18.38**
	ROUGE-L	71.8	63.33	**-8.47**
	CIDEr	2.53	1.10	**-1.43**

Table 1: Accuracy of our implementations versus the original paper's

4 Results & Analysis

Parameter and Compute Efficiency: Figure 3 shows our findings when comparing parameter and compute efficiency between FFT models and LoRA models. The most consistent finding across all experiments was the massive reduction in trainable parameters. LoRA reduced trainable parameters by over 99% in every setting. For example, full fine tuning RoBERTa required updating ~ 125 million parameters, while LoRA updated fewer than 900,000. Consequently, peak VRAM usage was reduced by approximately $3\times$, and average epoch time decreased notably, with the most significant speedups observed in MNLI and GPT-2 Small.

Task Performance:

- **SST-2 & CIFAR-10:** LoRA proved highly effective on simpler classification tasks. On SST-2, our LoRA implementation slightly matched or exceeded FFT accuracy. On CIFAR-10 with ViT, LoRA also remained remarkably close to FFT accuracy, successfully proving that LoRA generalizes well to computer vision tasks.
- **MNLI:** On this more complex reasoning task, we observed a minor accuracy drop compared to FFT, but our overall results are comparable.
- **E2E NLG (GPT-2):** Our generation metrics (BLEU, NIST, METEOR, ROUGE-L, CIDEr) show a larger gap from the paper's reported baselines for both FFT and LoRA. However, our METEOR scores are higher than the paper reported. There are a variety of reasons for this discrepacny, including unspecified hyperparameters or different prompt formats for training. We also suspect the original paper might have used used the official E2E evaluation toolkit with the Java METEOR implementation, whereas our scores are computed via HuggingFace's `evaluate` library, which uses NLTK's METEOR. The format of the multi-reference evaluation might be different across these two libraries, which may play a part in the inconsistency. We also found that applying the dropout described in the original paper degraded LoRA performance, so we report results without it. Despite the absolute score gap, the margin between FFT and LoRA remains small, supporting the paper's central claim that LoRA achieves competitive performance with significantly fewer trainable parameters.

Analysis: Our results validate the premise that many downstream tasks possess a low intrinsic dimension and do not require updating the entire pre-trained model. However, the discrepancies in the GPT-2 generation tasks suggest that harder tasks and generative models are highly sensitive to hyperparameter tuning, rank choice (r), and layer targeting.

5 Reflections

Lessons Learned: The core takeaway is that LoRA is a highly practical, parameter efficient method when compute or memory is limited, provided a minor performance trade off is acceptable on complex tasks. We learned that implementation details, such as framework specific layer groupings (like Hugging Face's attention projections), present significant engineering hurdles. Furthermore, we realized that generation tasks require much finer hyperparameter tuning than basic classification tasks.

Future Work: Our primary limitation was insufficient compute to run comprehensive hyperparameter sweeps or multiple random seeds. Given more resources, future extensions would include experimenting with different LoRA ranks (r), injecting LoRA into the key (W_k) and output (W_o) weight matrices to see if it closes the gap in generation tasks, and comparing LoRA against other PEFT methods like Adapters or Prefix Tuning. Exploring token level tasks (e.g., NER) would also provide a more complete picture of LoRA's NLU capabilities.

References

[1] Alexey Dosovitskiy, Lucas Beyer, Alexander Kolesnikov, Dirk Weissenborn, Xiaohua Zhai, Thomas Unterthiner, Mostafa Dehghani, Matthias Minderer, Georg Heigold, Sylvain Gelly, Jakob Uszkoreit, and Neil Houlsby. An image is worth 16x16 words: Transformers for image recognition at scale. *CoRR*, abs/2010.11929, 2020.

[2] Conal Elliott. The simple essence of automatic differentiation (differentiable functional programming made easy). *CoRR*, abs/1804.00746, 2018.

[3] Edward J. Hu, Yelong Shen, Phillip Wallis, Zeyuan Allen-Zhu, Yuanzhi Li, Shean Wang, and Weizhu Chen. Lora: Low-rank adaptation of large language models. *CoRR*, abs/2106.09685, 2021.

[4] Yinhan Liu, Myle Ott, Naman Goyal, Jingfei Du, Mandar Joshi, Danqi Chen, Omer Levy, Mike Lewis, Luke Zettlemoyer, and Veselin Stoyanov. Roberta: A robustly optimized BERT pretraining approach. *CoRR*, abs/1907.11692, 2019.

[5] Alec Radford, Jeff Wu, Rewon Child, David Luan, Dario Amodei, and Ilya Sutskever. Language models are unsupervised multitask learners. 2019.